IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Etat du trafic en temps réel Rennes Métropole

SUBJECT: Advanced Big Data

AUTHORS:
ARAÚJO Joao
CÁCERES Alejandro

Date: 23/03/2026

Introduction and Context **2**
Theoretical Concept 2
Context and Selected Data 2
Data Pipeline Implementation **3**
Architecture Overview Diagram 3
Data Ingestion 3
Speed Layer 3
The Data Lake 3
Batch Layer 3
Data Storage Structuring **4**
Real-time Table Schema 4
Mapping Table Schema 5
Daily Statistics Schema 5
Data Visualization **5**
Real Time Traffic Map 5
Speed Analysis 6
Historical Daily Analytics 6
Conclusions and Challenges **7**

Introduction and Context

Theoretical Concept

We need to process information in seconds, but we also need our calculations to be 100% accurate over long periods of time. Doing both at the same time is very difficult.

To solve this, we use the Lambda Architecture. This architecture is a smart design that splits the data processing into two separate paths:

- The Speed Layer: This path processes the real-time data stream as soon as it arrives. It gives us immediate results, but it only looks at the newest data.
- The Batch Layer: This path processes all the data in batches. It is slower, but it looks at the entire history of the data.

An everyday example could be an application like Waze or Google Maps. When you are driving, the app uses the Speed Layer to tell you "There is a traffic jam right now 500 meters ahead." You need this information instantly to change your route. However, at the end of the month, city planners use the Batch Layer to calculate the "average travel time" on that road over the last 30 days.

Context and Selected Data

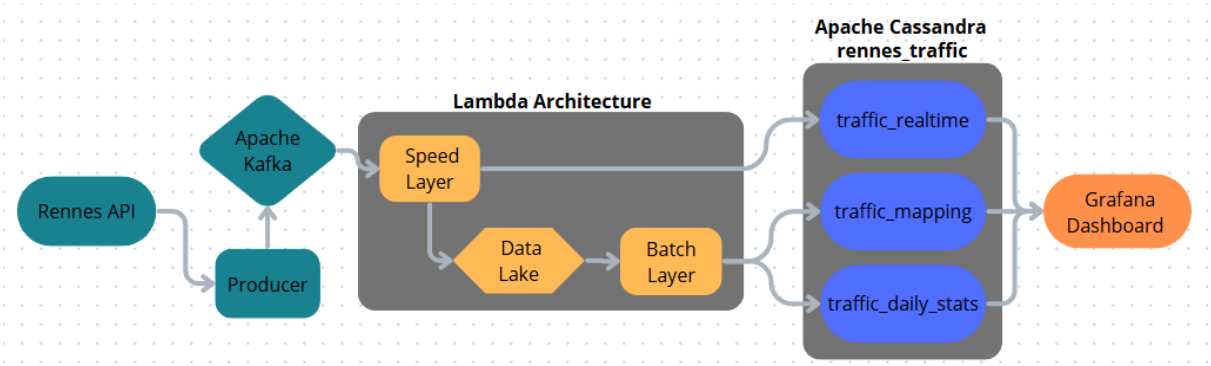
We are extracting information from the official Rennes Métropole API, which provides a real time data stream of the city's traffic conditions. Every few minutes, our system connects to this API to pull the latest information and feed it into our data pipeline.

To make our system efficient, we do not save all the data. Our system extracts only the key fields we need: the exact time the data was recorded (datetime), the name of the road (denomination), and the GPS coordinates (geo_shape) so we can place the street on a map. We also extract how fast the cars are driving right now (current_speed) and the maximum legal speed for that street (max_speed) and finally the predefined traffic density parameter (traffic_status)

Our main objective is to process this data to achieve two distinct goals. First, for immediate action, we want to show a real-time live map where streets turn colors like RED for 'congested' and GREEN for 'freeFlow', based on the predefined traffic status provided by the API. Second, to understand historical trends, we want to calculate the average daily speed for every street.

Data Pipeline Implementation

Architecture Overview Diagram



Data Ingestion

This is the first step of the project, where a Python script acts as the data extractor.It connects to the official Rennes Métropole REST API using an authenticated HTTP GET request. Since the API updates every 3 minutes, the script runs in a loop to fetch the newest data and transmits it directly to our Apache Kafka broker.

Speed Layer

This processing layer handles events exactly as they arrive using PySpark in Structured Streaming mode. During this phase, it extracts only the essential fields, translates French column names into English identifiers (such as converting vitesse_maxi to max_speed and denomination to street_name), and appends a new date_partition column using the current timestamp.

Once the transformations are complete, the system employs a dual output strategy to write the processed data into two different destinations simultaneously. First, it saves the records into a Cassandra database for the immediate visualizations. Then we also have a Data Lake for long term storage and future batch processing.

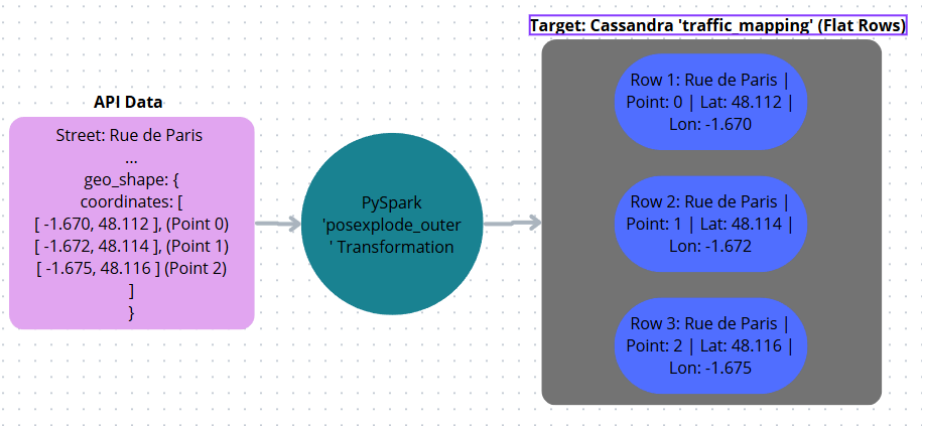
The Data Lake

While the Speed Layer writes real time data to Cassandra, it simultaneously archives all historical events into our local system. These files are systematically organized into folders based on the date_partition created in the previous step.

Batch Layer

This layer is responsible for handling heavy operations over the historical data and runs automatically every 9 minutes via a PySpark script. Unlike the streaming layer, it does not connect to Kafka; instead, it reads the historical Parquet files directly from the Data Lake.

This layer performs two main tasks. First, it solves a critical geographic problem. The API provides the maps as nested arrays of coordinates (geo_shape). The batch layer uses a PySpark posexplode transformation to "flatten" these arrays:



This creates a completely new row for every individual GPS point (latitude and longitude) along a street. It saves these precise coordinates into a Cassandra table named traffic_mapping. We intentionally moved this flattening task to the Batch Layer because, during testing, this multiplication process took too long to process all the streets, which broke the continuous flow of our Speed Layer.

Second, it calculates historical statistics. By grouping the historical data by street name and date, the batch layer computes the average daily speed. This allows us to compare current traffic conditions against historical trends. Mathematically, this is expressed as:

$$Average\ Daily\ Speed\ = \frac{1}{n}\sum_{i=1}^{n} v_i$$

Where v_i represents each recorded average speed value for that specific street throughout the day, and n is the total number of records. This mathematical aggregation allows us to compare current traffic against historical trends.

Data Storage Structuring

Our database needs to handle different types of information: real time updates and historical geographic data. For this, we use Cassandra, where we define three separate tables in a keyspace called rennes_traffic.

Real-time Table Schema

The traffic_realtime table is populated by the Speed Layer to store the latest speed and traffic status. Because Cassandra is a distributed database, a simple SQL "ORDER BY" would scan the entire network and destroy our latency. So we designed the schema using the street name as the Partition Key to ensure data lives on the same physical node. We also set date_time as the Clustering Column (in descending order), meaning the data is

already ordered. This design ensures that when Grafana requests current data, Cassandra returns the most recent row instantly.

Mapping Table Schema

On the other hand, the traffic_mapping table is generated by the Batch Layer to store the flattened GPS points required to draw the streets on a map. Its structure consists of precise latitude and longitude points, along with the sequence number of each coordinate. For this table, we include the street name and the road hierarchy, with date_time and point_order. This configuration allows the system to accurately reconstruct the exact geographic line of every street point by point.

Daily Statistics Schema

Finally, this table is also calculated by the Batch Layer. It stores the aggregated historical data, specifically the average daily speed for each street. The main calculation here is extracting the average speed (avg_speed) for each road.

Below is a summary of our defined data model:

Table Name	Target Layer	Key Fields
traffic_realtime	Speed Layer	avg_speed, traffic_status, max_speed, date_time
traffic_mapping	Batch Layer	latitude, longitude, point_order, street_name
traffic_daily_stats	Batch Layer	street_name, date_partition, aggregated avg_speed

Data Visualization

To visualize our results, we used Grafana. This tool connects directly to our Cassandra database and allows us to create charts easily.

We designed three main dashboards to show the different types of data we processed:

Real Time Traffic Map

This is our main screen. It uses the data from the Speed Layer to show a live map of Rennes. We added simple rules for the colors:

- Green: The traffic is fluid.
- Red: There is a traffic jam.

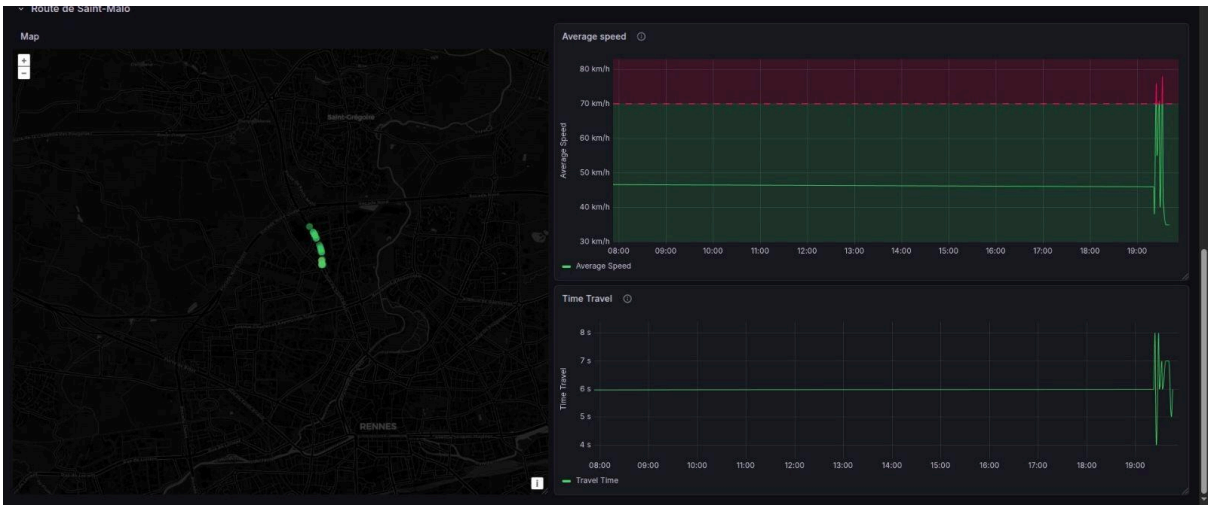
The map updates automatically every few seconds, so we can always see the current situation in the city.

Speed Analysis

The second dashboard helps us analyze specific streets in more detail. This graph helps us see exactly when the traffic usually slows down on the main roads.

Historical Daily Analytics

Finally, the third dashboard shows the results from our Batch Layer. Instead of showing live data, this table displays the average daily speed for only one street. This is useful to identify which streets have chronic traffic problems over the long term and also explore the comportment during the day.



Conclusions and Challenges

In conclusion, we successfully created a working Lambda Architecture. The system can handle real time data and process historical maps at the same time.

However, we faced a major challenge: hardware limits. The batch processing in Spark, especially when using the posexplode function to extract the GPS points from the maps, needs a lot of memory, or use two workers. Because our computers have limited RAM, we had to carefully adjust the Spark memory settings just to keep the program from crashing.

We learned that if we want to run this project in a real world company, we will definitely need better optimization and much more powerful computers (something we hope to finally get during our internships)